ASSIGNMENT 1

Text Preprocessing

Sr. No.	Name of Student	PRN	CO Mapped
1	Dhawse Spandan Yashwant	23070122085	CO1
2	Garvit Tyagi	23070122094	CO1

CA3 Project Title	RedactIQ: Explainable PII Detection and Redaction with Confidence-Scored Audit Trails

Aim

To perform text preprocessing using the NLTK library by implementing tokenization, regular expressions, stop-word removal, stemming, and lemmatization for preparing text data for Natural Language Processing applications.

The techniques are applied to a sample document from the corpus of the CA3 project RedactIQ, which detects and redacts personally identifiable information (PII) from unstructured text.

Learning Outcome:

After completing this experiment, students will be able to:

- Implement text preprocessing techniques such as tokenization, regular expression-based text cleaning, stop-word removal, stemming, and lemmatization using the NLTK library.
- Analyze and compare the effects of different text preprocessing techniques on the quality of textual data for Natural Language Processing tasks.
- Prepare cleaned and preprocessed text data suitable for downstream Natural Language Processing and machine learning applications.

Theory

Natural Language Processing (NLP) is the branch of artificial intelligence that enables computers to read, interpret and generate human language. Its main applications include machine translation, sentiment and opinion mining, conversational agents, and information extraction and de-identification, which is the task addressed by RedactIQ.

Text preprocessing is required because raw text contains punctuation, inconsistent capitalisation, URLs, digits and grammatical variants of the same word. Left untreated these inflate the vocabulary and make identical concepts appear as distinct features, so preprocessing converts raw text into a normalised sequence of meaningful units from which numerical features can be derived.

Tokenization splits text into smaller units called tokens. Sentence tokenization divides a document into sentences using the trained Punkt model, which is harder than splitting on the full stop because a full stop also occurs in abbreviations and e-mail addresses. Word tokenization then divides a sentence into words and punctuation, so “Please contact Rohan Mehta.” becomes ['Please', 'contact', 'Rohan', 'Mehta', '.'].

A regular expression is a formal pattern describing a set of strings, matched against the input in a single pass. It is used here for cleaning, where \d+ removes numbers, [^\w\s] removes punctuation and \s+

collapses repeated spaces, and also for detection, since structured PII such as e-mail addresses, telephone numbers and PAN numbers obey strict formats and can be located exactly by a pattern.

Stop words are very frequent function words such as the, is, at, on and to. They occur in almost every document and so carry little information for distinguishing one document from another, and removing them reduces dimensionality, so that a list such as please, contact, rohan, at, the, studies is reduced to please, contact, rohan, studies.

Stemming reduces an inflected word to a root form by applying fixed suffix-stripping rules. The Porter Stemmer uses five ordered phases of such rules, for example SSES → SS and IES → I; it consults no dictionary, so it is fast but the result need not be a real word, as in studies → studi and flies → fli.

Lemmatization also reduces a word to a base form, but uses the WordNet dictionary together with the word's part of speech, so the resulting lemma is always a valid word. The two therefore differ in both method and output: studies gives studi under stemming but study under lemmatization, and was gives wa against be. Stemming is preferred where speed matters, lemmatization where linguistic correctness matters.

Algorithm

1. Import the re and textwrap modules and the required NLTK components: sent_tokenize, word_tokenize, stopwords, PorterStemmer and WordNetLemmatizer.
2. Download the NLTK data packages punkt, punkt_tab, stopwords and wordnet.
3. Define the input document and a dictionary of regular-expression patterns for the PII categories EMAIL, PHONE, PAN and DATE.
4. Apply sentence tokenization to the raw text and display the resulting sentences.
5. Apply word tokenization to the raw text and display the token count and the first twelve tokens.
6. Search the raw text with each PII pattern and record the label, the character span and the matched string of every occurrence.
7. Clean the text with regular expressions by removing URLs, digits and punctuation and by collapsing repeated whitespace.
8. Tokenize the cleaned text, convert it to lower case, retain alphabetic tokens only, and remove the tokens present in the NLTK English stop-word list.
9. Apply the Porter Stemmer and the WordNet Lemmatizer to each remaining token and display the two results side by side.
10. Display a set of test words for which the stem and the lemma differ, so that the two techniques can be compared.

Program

```python
import re
import textwrap
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)

# Sample document from the RedactIQ corpus
text = ("Please contact Rohan Mehta at rohan.mehta@sitpune.edu.in or on "
        "+91-9876543210. His PAN is ABCDE1234F and the studies were "
        "uploaded to https://portal.sitpune.edu.in/records on 14/10/2026.")

# Patterns used to locate personally identifiable information
PII_PATTERNS = {
    "EMAIL": r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
    "PHONE": r"(?:\+91[-\s]?)?[6-9]\d{9}",
    "PAN":   r"\b[A-Z]{5}\d{4}[A-Z]\b",
    "DATE":  r"\b\d{1,2}[/-]\d{1,2}[/-]\d{2,4}\b",
}
```

```python
def show(label, items):
    """Print a token list wrapped to the width of the page."""
    print(textwrap.fill(f"   {label} : {items}", width=88,
                        subsequent_indent=" " * 14))

def clean(t):
    """Remove URLs, digits, punctuation and extra spaces using regex."""
    t = re.sub(r"https?://\S+", " ", t)      # URLs
    t = re.sub(r"\d+", " ", t)               # numbers
    t = re.sub(r"[^\w\s]", " ", t)           # punctuation
    return re.sub(r"\s+", " ", t).strip()    # extra whitespace
```

```python
# ---------------- Step 1 : Sentence tokenization ----------------
sentences = sent_tokenize(text)
print("1. SENTENCE TOKENIZATION :", len(sentences), "sentences")
for s in sentences:
    print(textwrap.fill(s, width=86, initial_indent="   | ",
                        subsequent_indent="     "))

# ---------------- Step 2 : Word tokenization --------------------
tokens = word_tokenize(text)
print("\n2. WORD TOKENIZATION :", len(tokens), "tokens")
show("first 12", tokens[:12])

# ---------------- Step 3 : Regular expressions ------------------
print("\n3. REGULAR EXPRESSIONS")
print("   (a) PII extracted before cleaning :")
for label, pattern in PII_PATTERNS.items():
    for m in re.finditer(pattern, text):
        print(f"       {label:<6} [{m.start():>3}:{m.end():<3}] {m.group()}")
cleaned = clean(text)
print("   (b) Cleaned text :")
print(textwrap.fill(cleaned, width=88, initial_indent=" " * 7,
                    subsequent_indent=" " * 7))
```

```python
# ---------------- Step 4 : Stop-word removal --------------------
stop_words = set(stopwords.words("english"))
words = [w.lower() for w in word_tokenize(cleaned) if w.isalpha()]
filtered = [w for w in words if w not in stop_words]
print("\n4. STOP-WORD REMOVAL :", len(words), "->", len(filtered), "tokens")
show("before ", words)
show("after  ", filtered)
show("removed", [w for w in words if w in stop_words])

# ---------------- Step 5 : Stemming and lemmatization -----------
ps, wnl = PorterStemmer(), WordNetLemmatizer()
print("\n5. STEMMING vs LEMMATIZATION")
seen = list(dict.fromkeys(filtered))          # keep order, drop repeats
print(f"   {'Token':<14}{'Porter stem':<16}{'WordNet lemma'}")
for w in seen:
    print(f"   {w:<14}{ps.stem(w):<16}{wnl.lemmatize(w)}")

demo = ["studies", "studying", "was", "caring", "flies"]
print("\n   Cases where the two differ :")
print(f"   {'Token':<14}{'Porter stem':<16}{'WordNet lemma'}")
for w in demo:
    print(f"   {w:<14}{ps.stem(w):<16}{wnl.lemmatize(w, pos='v')}")
```

Output

```
1. SENTENCE TOKENIZATION : 2 sentences
   | Please contact Rohan Mehta at rohan.mehta@sitpune.edu.in or on +91-9876543210.
   | His PAN is ABCDE1234F and the studies were uploaded to
     https://portal.sitpune.edu.in/records on 14/10/2026.

2. WORD TOKENIZATION : 28 tokens
   first 12 : ['Please', 'contact', 'Rohan', 'Mehta', 'at', 'rohan.mehta', '@',
              'sitpune.edu.in', 'or', 'on', '+91-9876543210', '.']
```

```
3. REGULAR EXPRESSIONS
   (a) PII extracted before cleaning :
       EMAIL   [ 30:56 ] rohan.mehta@sitpune.edu.in
       PHONE   [ 63:77 ] +91-9876543210
       PAN     [ 90:100] ABCDE1234F
       DATE    [175:185] 14/10/2026
   (b) Cleaned text :
       Please contact Rohan Mehta at rohan mehta sitpune edu in or on His PAN is ABCDE F
       and the studies were uploaded to on
```

```
4. STOP-WORD REMOVAL : 24 -> 13 tokens
   before  : ['please', 'contact', 'rohan', 'mehta', 'at', 'rohan', 'mehta', 'sitpune',
              'edu', 'in', 'or', 'on', 'his', 'pan', 'is', 'abcde', 'f', 'and', 'the',
              'studies', 'were', 'uploaded', 'to', 'on']
   after   : ['please', 'contact', 'rohan', 'mehta', 'rohan', 'mehta', 'sitpune', 'edu',
              'pan', 'abcde', 'f', 'studies', 'uploaded']
   removed : ['at', 'in', 'or', 'on', 'his', 'is', 'and', 'the', 'were', 'to', 'on']
```

```
5. STEMMING vs LEMMATIZATION
   Token          Porter stem     WordNet lemma
   please         pleas           please
   contact        contact         contact
   rohan          rohan           rohan
   mehta          mehta           mehta
   sitpune        sitpun          sitpune
   edu            edu             edu
   pan            pan             pan
   abcde          abcd            abcde
   f              f               f
   studies        studi           study
   uploaded       upload          uploaded

   Cases where the two differ :
   Token          Porter stem     WordNet lemma
   studies        studi           study
   studying       studi           study
   was            wa              be
   caring         care            care
   flies          fli             fly
```

Explanation of the Output

Sentence tokenization returns two sentences; the full stops inside the e-mail address and the URL are correctly not treated as boundaries. Word tokenization gives 28 tokens, separating @ from the address but keeping +91-9876543210 whole, which shows the tokenizer works on general rules and not on knowledge of these formats.

The regular expressions locate all four PII items with their exact character spans. The cleaned text below them shows why the order matters: cleaning removes digits and punctuation, so ABCDE1234F becomes ABCDE F and the telephone number and URL disappear entirely. Had the text been cleaned first the PII would no longer have been recoverable. Stop-word removal then reduces 24 tokens to 13.

The final table contrasts the two normalisation methods. The Porter Stemmer maps studies to studi, was to wa and flies to fli, none of which is an English word, whereas the lemmatizer returns study, be and fly; for caring both return care, since there the suffix rule happens to produce a valid form.

Conclusion

In this experiment the standard text preprocessing pipeline was implemented using the NLTK library and applied to a sample document from the RedactIQ project. Sentence and word tokenization were performed, regular expressions were used both to clean the text and to extract structured personally identifiable information, stop words were removed, and the remaining tokens were normalised by stemming and by lemmatization. The results show that preprocessing substantially reduces the size of the token set, from 28 raw tokens to 13 content-bearing ones, while retaining the words that carry meaning, which lowers the dimensionality of the feature vectors built in the later experiments. The comparison of the two normalisation methods showed that the Porter Stemmer is faster but can produce forms that are not real words, whereas the WordNet Lemmatizer uses a dictionary and the part of speech to return a valid lemma. An important observation for the project is that the order of the operations matters, because regular-expression cleaning destroys exactly the strings that the PII detector needs, so extraction must precede cleaning. It was also noted that the original case-preserved text must be retained, since capitalisation is a primary cue for recognising person and organisation names. The preprocessed output produced here is therefore suitable for the feature extraction and vectorization stages of the subsequent assignments.